TECHNICAL DESIGN · CODING MAP · PRODUCTION

# Financial Statement Review Agent

What each service does · local vs prod · where GenAI sits

| Field | Value |
|---|---|
| Document | TECH-001 · Version 1.1 |
| Date | 9 September 2026 |
| Input | BRD / HLD / running codebase |

## 1. Coding map (MVC + services)

| Layer | Path | Does |
|---|---|---|
| View | `ui/app.py` | Sample review · upload UI · ops / alerts |
| Controller | `api/main.py` | REST: reviews, upload, health, feedback, alerts |
| Orchestrator | `src/services/review.py` | Calls services in order; saves run pack |
| Model | `src/models.py` | Pydantic DTOs (Finding, ReviewPack, …) |
| Store | `src/store.py` | Load/parse CSV · save uploads · log tail |
| Config | `src/config.py` | Env, thresholds, math rules |

## 2. Services — what each one does

| Service | File | Responsibility | GenAI? |
|---|---|---|---|
| Validation | `validation.py` | Match formulas (totals) + Balance Sheet identity (Assets = L+E). Emits pass/fail findings with expected/actual + evidence. | No |
| YoY | `yoy.py` | Compare current vs prior lines. Flags material % / absolute change. Headline lines become findings for comments. | No |
| Observation | `observation.py` | Turns findings into short review text. Uses OpenAI if key set; else evidence templates. Must not invent amounts. | **Yes** |
| Notify | `notify.py` | On complete / critical fail: write log, optional email, optional webhook. | No |
| Health | `health.py` | `GET /health` status. `trigger_down_alert()` for ops when service is down / demo alert. | No |
| Review | `review.py` | `build_review_pack`: validate → YoY → observe → notify → save JSON audit. | Calls observation |

> **GenAI used only here:** observation wording. Numbers and pass/fail always come from validation + YoY rules.

## 3. Local runtime pipeline

```
Upload / sample CSV
  → save_upload() → data/runtime/uploads/  (+ optional PDF → documents/)
  → parse_csv_bytes() → StatementBundle
  → validate_all() → findings
  → compare_yoy() → variances + YoY findings
  → write_observations() → GenAI or templates
  → notify_review_events() → notifications.log
  → save_pack() → data/runtime/runs/run-*.json
```

Optional PDF is stored for audit only — not OCR'd into numbers in v1.

## 4. Local storage map

| Path | Contents |
| --- | --- |
| `data/sample/` | Built-in demo CSVs |
| `data/runtime/uploads/` | Uploaded statement CSVs |
| `data/runtime/documents/` | Supporting PDFs (store only) |
| `data/runtime/runs/` | Review pack JSON (audit) |
| `data/runtime/notifications.log` | Complete / critical / health-down alerts |
| `data/runtime/feedback.jsonl` | Useful / not useful votes |

## 5. Production architecture (target)

Same services. Stronger plumbing for files, jobs, DB, and ops.

```
Client / ERP / UI
    → API  POST /reviews/upload
    → Save files to S3  (csv/ + documents/)
    → Enqueue review job  (SQS / Redis / similar)
    → Workers: parse → validate → YoY → observations
    → Save pack to Postgres (+ pack JSON to S3)
    → Notify  (email / webhook / Slack)
    → If GET /health fails → auto alert ops
       (PagerDuty / email / on-call)
```

### 5.1 Two CSVs uploaded — what does the client get?

| Mode | Behaviour | Response |
|---|---|---|
| **Local / demo (now)** | **Sync** — both CSVs required (current + prior). Pipeline runs in the same request. | Full `ReviewPack` JSON / UI tabs when done |
| **Prod · many live requests** | **Async** — accept upload fast; do not hold HTTP open for GenAI. | `202 Accepted` + `run_id` → poll `GET /reviews/{id}` or webhook push |

```
Many uploads at once
  → API saves files + enqueues job  (returns run_id immediately)
  → N workers process jobs in parallel
  → Pack saved → notify user/system
  → Client fetches pack by run_id
```

Same 2 CSVs and same services — only sync vs queue changes under load.

### 5.2 Local vs prod

| Concern | Local (now) | Production |
|---|---|---|
| Files | Disk under `data/runtime/` | S3 / Azure Blob / GCS |
| Review packs | JSON files | Postgres (+ S3 object) |
| Jobs | Sync in API/UI process | Queue + worker pods |
| Alerts | Log (+ optional SMTP) | Monitored health → Slack/email/PagerDuty |
| Secrets | `.env` | Secret Manager / Key Vault |
| API / UI | localhost | Containers + load balancer |

### 5.3 Main API contracts (integrate anywhere)

| Endpoint | Purpose |
|---|---|
| `POST /reviews` | Run review on sample/stored company ids |
| `POST /reviews/upload` | Upload current+prior CSV (+ optional PDF) |
| `GET /reviews/{id}` | Fetch audit pack |
| `GET /health` | Liveness for LB / monitors |
| `POST /alerts/health-down` | Ops / demo: fire downtime alert |
| `POST /feedback` | Observation useful / not useful |

## 6. Algorithms & optimizations (judge advantage)

We do **not** "search a PDF with AI" for totals. We use fast, deterministic algorithms first — that is the accuracy advantage.

### 6.1 Line lookup — O(1) dictionary, not search

> After parse, lines become a map `code → amount` (`lines_by_code`). Every formula looks up codes in **O(1)** average time — no scanning the whole sheet each time, no fuzzy text search.

### 6.2 Math engine — rule table (match formulas)

Preconfigured parent → children rules (example):

```
TOTAL_CA     = CASH + AR + INVENTORY
TOTAL_ASSETS = TOTAL_CA + FA
GROSS_PROFIT = REVENUE + COGS   (COGS negative)
NET_INCOME   = OPERATING_INCOME + INTEREST + TAX
…
```

**Algorithm:** for each rule, `expected = sum(children)`; compare to reported parent within `epsilon`. Complexity ~ **O(R + L)** where R = rules, L = lines — tiny and predictable.

### 6.3 Consistency — Balance Sheet identity

```
diff = TOTAL_ASSETS − (TOTAL_LIAB + EQUITY)
if |diff| > epsilon → FAIL CONS-01
```

One equation, hard fail — no ML needed.

### 6.4 YoY materiality — threshold algorithm

```
abs_chg = current − prior
pct_chg = abs_chg / |prior|     (guard prior ≈ 0)
material if |pct| ≥ threshold (default 20%)
         OR |abs_chg| ≥ absolute floor
```

**Optimization:** only *headline* codes (Revenue, Gross profit, Net income, Total assets, …) become observation findings — full variance list still kept for analysis. That cuts GenAI/template noise and cost.

### 6.5 Pipeline optimization

| Idea | Why it helps |
|---|---|
| Rules before GenAI | Numbers never hallucinated; LLM only explains findings |
| Validate ∥ YoY (prod workers) | Independent stages can run in parallel; observations wait for both |
| Async queue under load | API stays fast; GenAI latency does not block other users |
| Headline filter | Fewer observation calls → lower cost / cleaner review pack |
| Idempotency (prod) | Same company+years+file hash → reuse pack; avoid double GenAI spend |
| Evidence chips from findings | No second "search" step — evidence already attached to the check |

### 6.6 What to say in the demo (30 seconds)

> "Lookup is by line code map, not AI search. Math and Balance Sheet identity are formula algorithms. YoY uses a threshold rule. GenAI only writes comments on those results. Under traffic we queue jobs so many uploads get a run_id immediately and results are fetched when ready."

## 7. Tests (what we coded for quality)

- `tests/test_review.py` — clean pack passes; broken pack MATH + CONS fail; YoY revenue material; pack has trace; feedback + health
- CI: `.github/workflows/ci.yml` — install → ingest check → pytest → import API

## 8. One-line summary for judges

> We coded MVC services where rules own numbers and GenAI only writes observations; local sync returns the full pack; production queues many uploads, stores files in S3, runs workers with formula + threshold algorithms first, then notifies and alerts on health failure.